

(AI+嵌入式)：Day8-模型训练与量化

1. 课程目标：

1.1 主要内容

如何使用第七课已经产生的数据集，训练正弦波的TensorFlow 模型，并导出为 MCU 可部署的 TFLite 模型，并在已有的模型的基础上验证成功。

1.2 知识目标

1. 为什么这份脚本要这样写；
2. 每一段代码在整个 TinyML 流程里起什么作用；
3. 训练出来的 `.keras`、`.tflite`、`.h` 文件分别是什么；
4. 为什么一个 Python 脚本，最后能产出 MCU 工程里能直接使用的模型文件。

1.3 回答的几个问题

1. 为什么数据要 reshape 成 `[N, 1]`？
2. 为什么只对输入做归一化？
3. 为什么训练集、验证集、测试集要分开
4. 为什么导出两个 `.tflite` 模型？
5. 为什么 int8 更适合 MCU？
6. 为什么最后还要再转成 `.h` 头文件？

如果你后续想把模型放到 STM32、GD32、ESP32 或其他 MCU 上，这份脚本背后的思路都非常重要。

1.4 输出目标

脚本运行成功，并且可以将模型部署成功

x	y							
0	0.001873							
0.006289	-0.07365							
0.012579	0.044607							
0.018868	0.061327							
0.025158	-0.09193							
0.031447	0.112086							
0.037737	0.083467							
0.044026	0.105443							

0.050316	-0.00031						
0.056605	0.077778						
0.062895	0.088839						
0.069184	0.023501						
0.075474	0.023431						
0.081763	0.106371						
0.088053	0.178824						
0.094342	0.187069						
0.100632	0.039203						
0.106921	0.082094						
0.113211	0.236735						
0.1195	0.118372						
0.125789	0.113742						
0.132079	-0.03085						
0.138368	0.108143						
0.144658	0.040267						
0.150947	0.159817						
0.157237	0.152237						
0.163526	0.11969						
0.169816	0.177013						
0.176105	0.208388						
0.182395	0.211254						
0.188684	0.318329						
0.194974	0.150723						
0.201263	0.195269						
0.207553	0.291146						
0.213842	0.179117						
0.220132	0.32522						
0.226421	0.178282						
0.232711	0.319901						
0.239	0.211303						
0.24529	0.178929						
0.251579	0.231481						
0.257868	0.223256						
0.264158	0.257923						
0.270447	0.269705						
0.276737	0.278397						
0.283026	0.350531						
0.289316	0.233388						
0.295605	0.33586						
0.301895	0.23606						
0.308184	0.290523						
0.314474	0.376685						

 sine_raw - Copy.csv

2. 模型训练的流程



 train_sine_tensorflow.py

2.1 导入模块部分

先看文件最前面的导入部分：

代码块

```
1  import json
2  from pathlib import Path
3
4  import matplotlib.pyplot as plt
5  import numpy as np
6  import pandas as pd
7  import tensorflow as tf
```

这里导入了 5 个关键模块。

json 用来保存归一化参数。

Path 是 Python 里处理路径的一个工具。

它比传统字符串拼接路径更规范，尤其在 Windows、Linux、macOS 上都比较方便。

matplotlib.pyplot 是用来画图的。

numpy 数值计算的基础库。

后面数据切分、洗牌、生成曲线输入点、类型转换，都会大量用到它。

`pandas` 是用来读 CSV 的

你可以把它理解成 Python 里处理表格数据最常用的工具。

`tensorflow` 是核心

训练模型、定义网络、导出 TFLite，全都依赖它。

这里顺便提醒大家一个很重要的概念：

TensorFlow 负责训练。

TFLite / TFLite Micro 主要负责推理和部署。

很多刚接触 TinyML 的同学，容易把这几者混在一起。

你要把它们区分清楚：

TensorFlow 像一个大工厂，用来设计和训练模型；

TFLite 像一个压缩后的部署格式，用来在资源受限的平台上运行；

TFLite Micro 则是进一步面向 MCU 的推理运行时。

2.2 路径配置部分

接着看第一部分：

代码块

```
1 CSV_PATH = "sine_raw - Copy.csv"
2 OUT_DIR = Path("sine_tf_output")
3 OUT_DIR.mkdir(parents=True, exist_ok=True)
```

这一段代码做了两件事。

第一件事，告诉脚本：训练数据集在哪里。

这里默认读取的是 `sine_raw - Copy.csv`。

第二件事，创建一个输出目录 `sine_tf_output`。

后面所有训练产物，都会放到这个目录里。

如果这些文件随便散落在工程目录里，后期会非常乱。

2.3 固定随机种子

代码块

```
1 SEED = 42
2 np.random.seed(SEED)
3 tf.random.set_seed(SEED)
```

这段代码的作用，是让实验结果尽量可复现。

为什么需要它？

因为训练神经网络时，有很多地方都带随机性。

2.4 读取数据

代码块

```
1 df = pd.read_csv(CSV_PATH)
```

这句代码把 CSV 文件读进来，放到了一个 DataFrame 里。

你可以把 DataFrame 理解成 Python 里的电子表格。

有列名，有行数据，可以按列访问。

代码块

```
1 required_cols = {"x", "y"}
2 if not required_cols.issubset(df.columns):
3     raise ValueError(f"CSV 文件必须包含列: {required_cols}, 当前列为:
    {df.columns.tolist()}")
```

这段代码是在做字段检查。

它要求 CSV 必须包含两列：`x` 和 `y`。

2.5 缺失值检查

防止训练时出现隐蔽错误

代码块

```
1 if df[["x", "y"]].isna().any().any():
2     raise ValueError("CSV 中存在缺失值，请先清理数据。")
```

这里是在检查 `x` 和 `y` 列中是否存在空值，也就是缺失值。

2.6 数据转成 float32

```
代码块 = df["x"].to_numpy(dtype=np.float32)
2    y = df["y"].to_numpy(dtype=np.float32)
```

这里做了两件事：

第一，把 Pandas 里的列取出来；

第二，转成 NumPy 数组，并且指定类型为 `float32`。

为什么是 `float32`，不是 Python 默认的浮点类型？

原因有两个。

第一个原因，TensorFlow 训练中最常用的浮点类型就是 `float32`。

它在精度和资源开销之间比较平衡。

第二个原因，这个项目后面还要做部署。

如果一开始数据类型就比较规范，后续转换到 TFLite 时也更自然。

尤其是 TinyML 场景里，我们要尽量避免不必要的类型混乱。

对嵌入式工程师来说，可以这么理解：

`float32` 就相当于“我先用标准的 32 位浮点把训练流程规范起来”，等后面导出 int8 模型时，再进一步压缩。

2.7 为什么要 reshape 成 `[N, 1]`

代码块

```
1    x = x.reshape(-1, 1)
2    y = y.reshape(-1, 1)
```

这一步非常关键，也是很多初学者最容易忽略的地方。

假设原来的 `x` 是一维数组，形状可能是：

代码块

```
1    (1000,)
```

这个意思是：有 1000 个数，但它只是一个一维向量。

而神经网络在处理输入时，更习惯的格式是二维：

代码块

```
1    (1000, 1)
```

这表示：一共有 1000 个样本，每个样本有 1 个特征。

这里的 `-1` 表示这一维由 Python 自动推断。

所以 `reshape(-1, 1)` 的意思就是：

“请把数据整理成若干行、1 列。”

这个动作在机器学习里非常常见。

因为大多数模型都默认输入格式是：

样本数 × 特征数

在这个正弦项目里，每个样本只有一个输入特征，也就是 `x`。

所以特征数是 1。

同理，输出 `y` 也是一个连续值，所以它也被整理成 `[N, 1]`。

如果你未来做的是三轴加速度计手势识别，

那输入形状可能就不是 `[N, 1]`，而是 `[N, 时间窗长度, 通道数]`。

但这个正弦项目的本质思路是一样的：

先把样本维度和特征维度整理清楚。

2.8 打印数据基本信息

代码块

```
1 print("样本数量:", len(x))
2 print("x shape:", x.shape)
3 print("y shape:", y.shape)
4 print("x 范围:", float(x.min()), "->", float(x.max()))
5 print("y 范围:", float(y.min()), "->", float(y.max()))
```

为什么训练前必须看

看起来只是打印信息，但其实非常有价值。

训练前打印这些内容，有三个目的。

第一个目的，确认样本量。

你要知道自己到底在拿多少数据训练，不然训练结果好坏都没法判断。

第二个目的，确认 shape。

很多训练报错，本质上都是 shape 对不上。

比如模型要二维输入，你给了一维；

或者输出 shape 不符合损失函数要求。

第三个目的，确认数值范围。

正弦数据一般 x 可能在 0 到 2π 或更大范围，

y 通常在 -1 到 1 附近。

如果你发现 y 范围特别离谱，比如几百、几千，那就要怀疑数据是否错了。

所以这部分打印信息，并不是可有可无的“调试残留”，

而是训练前的数据体检。

2.9 划分数据集

2.9.1 把数据打乱

代码块

```
1 indices = np.arange(len(x))
2 np.random.shuffle(indices)
3
4 x = x[indices]
5 y = y[indices]
```

为什么要划分训练集、验证集、测试集

这里先生成所有样本的索引，然后把索引打乱，再按打乱后的顺序重新排列 x 和 y 。

为什么要打乱？

因为原始数据通常是按某种顺序排列的。

比如正弦数据可能就是从小到大按 x 排的。

如果不打乱，前 70% 的训练集、后 15% 的验证集、最后 15% 的测试集，就会对应不同区间的数据。这样模型评估会失真。

打乱之后，训练集、验证集、测试集都会更均匀地覆盖整个数据分布。

有的时候我们会看到，很多数据集本身就是错乱的，说明把数据打乱这个步骤是在产生数据集的时候，已经做好了，那么在训练的时候就不需要对数据做打乱的处理。

当然，在训练的时候重新对数据打乱，也是没问题的。

代码块

```
1 n_total = len(x)
2 n_train = int(n_total * 0.70)
3 n_val = int(n_total * 0.15)
```

2.9.2 这里定义了数据集划分比例：

70% 训练，15% 验证，15% 测试。

然后再切出来：

代码块

```
1 x_train = x[:n_train]
2 y_train = y[:n_train]
3
4 x_val = x[n_train:n_train + n_val]
5 y_val = y[n_train:n_train + n_val]
6
7 x_test = x[n_train + n_val:]
8 y_test = y[n_train + n_val:]
```

为什么要分成三份，而不是只分训练集和测试集？

这里要讲清三个集合的职责。

训练集，是给模型“学习”的。

也就是模型在这个数据上不断调整自己的权重。

验证集，不参与参数更新，它是训练过程中用来观察模型效果的。

比如 early stopping 就依赖验证集的 loss。

训练过程中，如果验证集 loss 连续很多轮都没有明显改善，就提前停止训练。

测试集，是最后做“最终考试”的。

它不应该参与训练过程中的任何决策。

否则你就等于偷看答案了。

对嵌入式工程师来说，可以这么类比：

训练集，相当于平时练习题；

验证集，相当于模拟考试，用来判断最近学得怎么样；

测试集，相当于正式考试，最后只看一次成绩。

2.10 输入归一化：

这一步对 MCU 部署非常关键

代码块

```
1 x_min = x_train.min()
2 x_max = x_train.max()
3
4 def normalize_x(x_input: np.ndarray) -> np.ndarray:
```

```
5         return (x_input - x_min) / (x_max - x_min)
```

这段代码做的是最小值-最大值归一化。

公式就是：

输入归一化值 = **当前输入减去训练集最小值，再除以训练集范围。**

这样处理之后， x 就会被压缩到大约 0 到 1 的范围。

为什么要做归一化？

因为神经网络训练时，如果输入数值范围过大或者不稳定，训练会更困难。

归一化后的输入更规整，模型更容易收敛。

这里有一个特别重要的细节：

归一化参数是从训练集算出来的。

也就是说：

代码块

```
1  x_min = x_train.min()
2  x_max = x_train.max()
```

而不是从整个数据集去算。

为什么？

因为验证集和测试集应该模拟“未知数据”。

如果你在计算归一化参数时已经看到了它们，那严格来说就有信息泄露。

这个细节在很多教程里会被忽略，但它其实是规范机器学习流程的重要部分。

代码块

```
1  x_train_norm = normalize_x(x_train).astype(np.float32)
2  x_val_norm = normalize_x(x_val).astype(np.float32)
3  x_test_norm = normalize_x(x_test).astype(np.float32)
```

训练集、验证集、测试集都使用同样的归一化公式。

这里我要强调一个部署上的关键原则：

训练端怎么做预处理，部署端就必须怎么做。

比如你在 Python 中把 x 从原始范围映射到了 0 到 1，

那 MCU 端送进模型前，也必须做同样处理。

如果不做，模型在部署端的输入分布就和训练时不一致，输出就会不正确。

2.11 为什么要把归一化参数保存成 JSON

代码块

```
1 norm_params = {
2     "x_min": float(x_min),
3     "x_max": float(x_max),
4     "formula": "x_norm = (x - x_min) / (x_max - x_min)"
5 }
6 with open(OUT_DIR / "normalization.json", "w", encoding="utf-8") as f:
7     json.dump(norm_params, f, ensure_ascii=False, indent=2)
```

这一步非常好，因为它把训练时使用的归一化参数保存下来了。

这有什么意义？

因为后面你在 STM32 上部署时，不能靠“记忆”去还原这些参数。

要有明确、可追溯的文件。

比如这个 JSON 里会写清楚：

训练用的最小值是多少；

最大值是多少；

公式是什么。

这样你在 MCU 代码里就可以照着实现。

这一点看起来细，但实际上很像真实产品开发。

真正的工程不是“我大概记得”，而是“参数被明确记录”。

2.12 构建模型

代码块

```
1 model = tf.keras.Sequential([
2     tf.keras.layers.Input(shape=(1,), name="input_x"),
3     tf.keras.layers.Dense(16, activation="relu"),
4     tf.keras.layers.Dense(16, activation="relu"),
5     tf.keras.layers.Dense(1, name="output_y")
6 ])
```

这里开始正式定义神经网络。

先解释 `Sequential`。

它表示这是一个按顺序堆叠的模型。

也就是说，输入先经过第一层，再经过第二层，再经过第三层。

这对初学者非常友好，因为结构直观。

再看每一层。

2.12.1 第一层：

代码块

```
1 tf.keras.layers.Input(shape=(1,), name="input_x")
```

这表示输入张量的形状是 `(1,)`，也就是每个样本只有 1 个输入值。

这里对应的就是正弦函数里的 `x`。

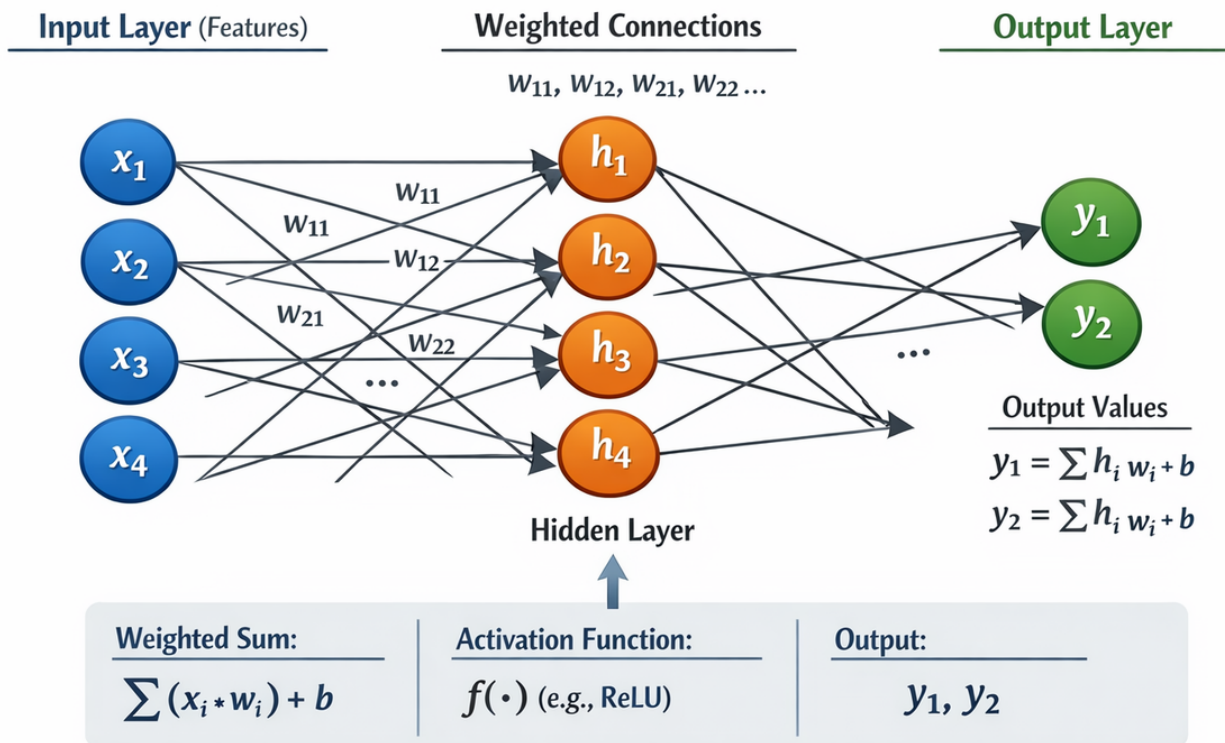
2.12.2 两层隐藏层

代码块

```
1 tf.keras.layers.Dense(16, activation="relu")
2 tf.keras.layers.Dense(16, activation="relu")
```

`Dense` 是全连接层。

Fully Connected Layer



意思是：这一层的每个神经元，都会和上一层所有输出相连。他对应的是矩阵的乘法和加法，可以理解为：

$$y = Wx + B$$

这里每层有 16 个神经元。

对于正弦函数这样的小任务，这个规模已经足够了。

激活函数用了 `relu`。

它的公式可以简单理解为：

输入大于 0，就原样输出；

输入小于等于 0，就输出 0。

ReLU 的作用不是做加权，而是对加权后的结果做一次处理：

- 如果结果没有意义或者不够强，压成 0
- 如果结果是有效的正响应，就保留下来

ReLU 正好符合这个思路：

- 输入结果大于 0，神经元输出正值，相当于“被激活”
- 输入结果小于等于 0，输出 0，相当于“没有激活”

所以它被叫做 **激活函数**。

这里的“激活”，本质上就是：

决定这个神经元是否对后面的网络继续产生影响。

为什么用 ReLU？

因为它简单、训练稳定、计算量小，是目前最常用的隐藏层激活函数之一。

2.12.3 最后一层

代码块

```
1 tf.keras.layers.Dense(1, name="output_y")
```

输出只有 1 个值，因为这个任务是回归任务。

输入一个 x ，输出一个连续值 y 。

因为我们这里输出的是连续的值，所以就不需要再加激活函数。

2.13 为什么这个任务是回归，不是分类

这里顺便要专门讲一下。

很多初学者会问：这个正弦波模型到底是什么类型的模型？

答案是：**回归模型**。

因为它预测的是一个连续值。

输入 x ，输出的是接近 $\sin(x)$ 的一个实数。

如果是分类任务，比如判断“上楼、下楼、静止”，输出通常是类别。

那最后一层设计和损失函数就会不同。

所以你要建立一个基本判断标准：

输出是连续数值，就是回归；

输出是离散类别，就是分类。

正弦波 Hello World 通常被用作回归入门案例，

因为它结构简单、可视化直观、又足够能说明模型拟合的核心思想。

2.14 编译模型：优化器、损失函数、评估指标

代码块

```
1 model.compile(  
2     optimizer=tf.keras.optimizers.Adam(learning_rate=0.01),  
3     loss="mse",  
4     metrics=["mae"]  
5 )
```

这一步叫“编译模型”。

你可以理解为：

定义好了网络结构之后，还要告诉 TensorFlow：

2.14.1 优化器

训练时怎么优化；

用什么标准判断误差；

训练过程中额外显示哪些指标。

这里用了 `Adam` 优化器，学习率是 `0.01`。

优化器的作用，是根据当前误差去更新模型权重。

你可以把它理解成“参数调整规则”。

`Adam` 是一个非常常用的优化器。

它通常比最基础的 SGD 更容易收敛，尤其适合初学者和中小模型。

2.14.2 损失函数：

代码块

```
1 loss="mse"
```

`MSE` 是均方误差。

意思是：把预测值和真实值的差先平方，再求平均。

为什么回归常用 MSE？

因为它能直接反映预测值偏离真实值的程度。

预测越准，MSE 越小。

2.14.3 指标

代码块

```
1 metrics=["mae"]
```

MAE 是平均绝对误差。

就是预测值和真实值之差取绝对值，再求平均。

为什么既要 MSE，又要 MAE？

因为 MSE 对大误差更敏感，

MAE 更直观，单位和原始输出一致。

二者结合起来看，会更全面。

简单说：

MSE 更适合拿来做优化目标；

MAE 更适合辅助理解误差大小。

2.15 model.summary() 到底在看什么

代码块

```
1 model.summary()
```

这句会打印模型结构摘要。

每一层的名字；

每一层输出 shape；

每一层参数个数；

总参数量。

这个正弦模型的参数量会非常小。

这恰恰是 TinyML 入门案例的价值之一。

因为它让你先理解整个流程，而不是一上来就面对几十万参数的大模型。

这里也可以顺带向观众强调：

模型不是越大越好。

在 MCU 场景里，模型大小、推理时间、RAM 占用、Flash 占用都很关键。

所以一个小而够用的模型，往往比一个很大但难部署的模型更有工程价值。

2.16 训练模型：为什么还要用 EarlyStopping

代码块

```
1  callbacks = [  
2      tf.keras.callbacks.EarlyStopping(  
3          monitor="val_loss",  
4          patience=30,  
5          restore_best_weights=True  
6      )  
7  ]
```

先解释 `EarlyStopping`。

它的意思是：

训练过程中，如果验证集 loss 连续很多轮都没有明显改善，就提前停止训练。

这里监控的是 `val_loss`。

也就是验证集上的损失。

`patience=30` 表示容忍 30 个 epoch 没改善。

如果 30 个 epoch 都没有变好，就停。

`restore_best_weights=True` 表示停止后，把模型权重恢复到“验证集效果最好”的那一轮，而不是最后一轮。

为什么这个机制很有必要？

因为训练轮数设得很大时，模型可能在训练集上越来越好，但在验证集上反而开始变差。

这就是过拟合的一种表现。

有了 `EarlyStopping`，训练会更稳，也更省时间。

对于嵌入式工程师来说，可以这样理解：

不是训练时间越长越好。

而是要在“学到规律”和“不过度死记硬背”之间找到平衡。

2.17 `model.fit()`：这是模型真正开始学习的地方

代码块

```
1  history = model.fit(  
2      x_train_norm,  
3      y_train,  
4      validation_data=(x_val_norm, y_val),  
5      epochs=300,  
6      batch_size=32,
```

```
7     verbose=1,  
8     callbacks=callbacks  
9 )
```

这一段是整个训练的核心。

先看输入：

`x_train_norm` 是归一化后的训练输入。

`y_train` 是真实标签，也就是对应的正弦值。

再看 `validation_data`。

这里给了验证集数据，让 TensorFlow 在每轮训练结束后都评估一次验证效果。

`epochs=300` 表示最多训练 300 轮。

epoch = 把所有训练数据完整地过一遍

- 训练时设置 `epochs=10`，意思就是**把全部数据从头到尾训练 10 次**。
- 每跑完一轮，就叫完成了 1 个 epoch。
- 但由于有 EarlyStopping，实际上通常不会真的跑满 300 轮。

`batch_size=32` 表示每次拿 32 个样本作为一个小批次来更新参数。

如果训练集有 700 个样本，batch size 是 32，

那么一个 epoch 里大约会分成很多个 batch 来训练。

$700/32=22$

2.18 模型是怎么“学会”正弦函数的

很多人第一次接触神经网络时，会觉得很神秘。

其实正弦项目非常适合把这个过程讲清楚。

模型一开始并不知道 `sin(x)` 是什么。

它只知道：

我输入一个数；

输出一个数；

然后拿输出和真实答案去比较；

如果差得多，就调整内部参数。

这个“内部参数”，就是每层神经元之间的权重和偏置。

经过很多轮训练后，模型逐渐找到了一套参数组合，

使得输入某个 `x` 时，输出值尽量接近真实 `sin(x)`。

所以它不是“背公式”，

而是通过大量样本，把一条近似曲线学出来。

这也是为什么神经网络最后可以拟合出一条类似正弦的曲线。

2.19 模型评估

训练完不能只看训练集

代码块

```
1 test_loss, test_mae = model.evaluate(x_test_norm, y_test, verbose=0)
2 print(f"测试集 MSE: {test_loss:.8f}")
3 print(f"测试集 MAE: {test_mae:.8f}")
```

这里是在测试集上做最终评估。

为什么必须有这一步？

因为训练集上的效果好，并不代表模型真的学会了。

它可能只是记住了训练集。

只有在没参与训练的测试集上仍然表现不错，才说明它有一定泛化能力。

这里输出两个指标：

测试集 MSE；

测试集 MAE。

如果这两个值都比较小，说明模型在未见过的数据上也能较好地拟合正弦曲线。

这部分视频里可以重点强调一句：

训练结果不能只看训练集，最终一定要看测试集。

2.19.1 保存 `.keras` 模型

代码块

```
1 model.save(OUT_DIR / "sine_model.keras")
```

这里保存的是 Keras 原生模型文件。

这个文件里包含什么？

它包含：

网络结构；

训练好的权重；

以及与模型有关的元信息。

简单理解，它是“训练后的完整模型存档”。

为什么先保存 `.keras` 很有意义？

因为它是一个比较完整、方便后续继续使用的格式。

你以后想继续训练、重新评估、再导出别的格式，都可以基于它。

但是，这个格式通常不直接用于 MCU 部署。

MCU 更常见的是 `.tflite`，或者进一步转成 C 数组。

所以 `.keras` 更像是“训练侧的标准保存格式”。

2.20 导出 float32 TFLite

代码块

```
1 converter = tf.lite.TFLiteConverter.from_keras_model(model)
2 tflite_model = converter.convert()
```

这是部署前的第一步

这里把 Keras 模型转换成 TFLite 模型。

你可以把这个动作理解成：

把训练侧的大模型表示，转换成更适合部署的轻量格式。

然后：

代码块

```
1 float_tflite_path = OUT_DIR / "sine_model_float32.tflite"
2 with open(float_tflite_path, "wb") as f:
3     f.write(tflite_model)
```

这里把转换结果保存成一个二进制文件，也就是 `.tflite`。

为什么叫 `float32.tflite`？

因为这个版本的模型仍然主要使用浮点表示，没有做 int8 量化。

这个版本的好处是：

转换简单；

数值行为通常更接近训练时；

适合先做功能验证。

但它的缺点也很明显：

在 MCU 上通常占空间更大；

推理效率可能不如 int8；

对没有硬件浮点优势的芯片不够友好。

所以这个脚本后面还会再导出一个更适合 MCU 的 int8 版本。

2.21 为什么 int8 更适合 MCU

先看代表性数据集函数：

代码块

```
1 def representative_dataset():
2     for i in range(min(200, len(x_train_norm))):
3         yield [x_train_norm[i:i+1]]
```

这个函数的作用，是给量化器提供一部分“有代表性”的输入样本。

为什么要有它？

因为在把浮点模型转换成 int8 模型时，

系统需要知道：实际输入数据大概落在什么范围。

这样它才能合理地把浮点值映射到 int8 的数值范围里。

这里选的是训练集归一化后的数据。

这是合理的，因为模型在真实部署时，输入也应该是同样归一化后的数据。

代码块

```
1 converter_int8 = tf.lite.TFLiteConverter.from_keras_model(model)
2 converter_int8.optimizations = [tf.lite.Optimize.DEFAULT]
3 converter_int8.representative_dataset = representative_dataset
4 converter_int8.target_spec.supported_ops =
  [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
5 converter_int8.inference_input_type = tf.int8
6 converter_int8.inference_output_type = tf.int8
```

2.22 把 .tflite 转成 C 数组

代码块

```
1 def convert_tflite_to_c_array(tflite_path: Path, output_header_path: Path,
    array_name: str):
2     data = tflite_path.read_bytes()
```

这里先把 `.tflite` 文件按二进制方式读进来。

然后：

代码块

```
1 f.write(f"const unsigned char {array_name}[] = {{\n")
2 ...
3 f.write(f"const unsigned int {array_name}_len = {len(data)};\n\n")
```

这里把模型二进制数据一字节一字节写成 C 数组形式。

最终你会得到一个头文件，例如：

代码块

```
1 const unsigned char g_sine_model_data[] = { ... };
2 const unsigned int g_sine_model_data_len = ...;
```

为什么这样 MCU 就能用了？

因为从本质上讲，`.tflite` 模型文件就是一段二进制数据。

在 PC 上它是存成文件；

在 MCU 工程里，如果不方便文件系统读取，那就可以直接把这段数据编进程序里。

也就是说：

在 PC 上，模型以文件形式存在；

在 MCU 上，模型可以以内存数组形式存在。

TFLM 或其他推理框架拿到这段模型数据后，就能去解析里面的网络结构、权重、量化参数等信息。

所以这个步骤是“从文件世界，进入嵌入式代码世界”的关键桥梁。